Pipelines ETL/ELT

LegalDataPlatform

Humberto Henriquez — Senior Data Engineer / Solutions Architect
Generado 2026-04-24

Pipelines ETL/ELT

Diseño

Cada pipeline sigue el mismo esqueleto:

```
Extract → Bronze → Normalize+Validate → [Quarantine] → Load → Gold
```

Los bloques están decorados con `@task` de Prefect; el flow orquestador los compone.

Extractores implementados

`RestAPIExtractor`

Aspecto	Implementación
Paginación	offset / cursor / page (configurable)
Retry	tenacity, exponential backoff, 5 intentos
Rate limiting	respeta `Retry-After` header, sleep asincrónico
Auth	headers (Bearer, API-Key) en config
Timeout	30s por request (configurable)
Observability	structlog + Prometheus counter por request

`CSVExtractor`

Aspecto	Implementación
Engine	Polars `scan_csv` (lazy)
Streaming	chunks configurables (default 10K rows)
Detección de schema	`infer_schema_length=1000`
Encoding	UTF-8 por defecto, configurable

`PostgresExtractor`

Aspecto	Implementación
Patrón	watermark-based incremental
Watermark store	tabla `pipeline_watermarks` (autocreada)
Idempotencia	re-ejecutar no duplica; sólo trae filas nuevas
Batch size	configurable (default 5000)

`S3ParquetExtractor`

Aspecto	Implementación
Engine	pyarrow `ParquetFile.iter_batches`
Column pruning	lista opcional de columnas
Compatibilidad	funciona contra MinIO local y S3 real vía endpoint

Normalización

Tres pasos para cada batch:

1. **Mapping source → canonical**: funciones `_map_*_csv()` traducen nombres de columna heterogéneos.
2. **Pydantic validation**: schemas en `src/schemas/` definen tipos, rangos, regex, normalización.
3. **DQ rule engine**: reglas declarativas en YAML.

DQ como código

Las reglas viven en `src/data_quality/rules/*.yaml`. Ventajas:

- Versionadas en git → historial de cambios de calidad.
- Revisables en PRs → el business analyst puede proponer nuevas reglas.
- Reutilizables → misma suite entre dev/staging/prod.

Los resultados se exponen como métricas Prometheus (`ldp_dq_checks_total{suite, result}`), y se integran con alertas.

Cargas

PostgreSQL

Pipeline: batch normalizado → `COPY` a temp staging → `INSERT ... ON CONFLICT ... DO UPDATE` → commit.

Throughput medido: ~150K rows/s en hardware m5.large equivalente, con 3 índices y FK constraints.

S3 Parquet

Pipeline: batch → Polars DataFrame → PyArrow Table → `pq.write_to_dataset(partition_cols=[...])`.

Ventajas de particionado por `ingestion_date`:
- Consumidores (Athena, Redshift Spectrum) aplican predicate pushdown.
- Retention por día es un simple `delete_prefix()`.

Idempotencia

Cada entidad tiene una **clave única de negocio**:

Entidad	Unique key
`legal_documents`	`(source_system, source_id, document_date)`
`transactions`	`(source_system, reference, transaction_date)`
`counterparties`	`(external_id)`
`contracts`	`(contract_number)`

`ON CONFLICT ... DO UPDATE` sobre esas keys → pipeline re-runnable sin riesgo de duplicados.

Orquestación con Prefect

Estructura de un flow

```python
@flow(name="legal-ingestion-flow")
async def legal_ingestion_flow(file_path: str, rules_path: str) -> dict:
    batches = await extract_from_csv(file_path, ...)
    await persist_to_bronze(batches, ...)
    valid, rejected = normalize_and_validate(batches, ...)
    if rejected:
        await persist_quarantine(rejected, ...)
    loaded = await load_legal_documents(valid, ...)
    await refresh_gold()
    return {"extracted": ..., "loaded": loaded, ...}
```

Retries y timeouts

```
@task(retries=3, retry_delay_seconds=30, log_prints=True)
```

Prefect re-ejecuta el task al fallar. El decorador acepta `retry_jitter_factor` para evitar thundering herd.

Deployment

`prefect deployment` asocia el flow a:
- un **work pool** (local, Docker, Kubernetes, ECS).
- una **schedule** (cron, interval, rrule).
- **parameters** default.

Registrados en deployments.py.

Estrategia de errores

Tipo	Respuesta
Error transitorio (red, 5xx)	retry con backoff
Rate limit (429)	sleep por `Retry-After`
Validación Pydantic	registro a quarantine, pipeline sigue
DQ rule severity=error	AssertionError → flow falla
DQ rule severity=warning	métrica incrementada, flow sigue
Error de integridad DB	flow falla, rollback, alerta
Lambda failure	SQS redrive → DLQ después de 5 intentos

Monitoreo y alertas

Dashboards Grafana recomendados:

1. **Throughput**: `rate(ldp_records_loaded_total[5m])` por pipeline.
2. **Calidad**: `rate(ldp_records_quarantined_total[5m])` — alarma si >5% por 10min.
3. **Latencia**: histograma `ldp_pipeline_duration_seconds` p95 por stage.
4. **DLQ**: `ApproximateNumberOfMessagesVisible` de SQS DLQ — alarma si >0.
5. **DB health**: conexiones activas, pg_stat_statements top 10, bloat % por tabla.